

# Dobot Magician E6 ultrasound teleoperation

How to connect, run, drive and pack down the arm for live probe-on-phantom demonstrations. Written for engineers who have not worked with a robot arm before.

**The arm is position controlled and does not feel you.** It will not stop for your hand, the bench, or the phantom.

- Hold **L1** or nothing moves. Release it and the arm stops — that is your first reflex, not the emergency stop.
- Keep the **red emergency stop** on the base within reach. Press to stop, rotate to release.
- The **robot base is the centre** of the workspace: the arm reaches 450 mm in every direction, **front and back**. Nobody puts a hand inside it while it is running.

---

Robot Dobot Magician E6  
Controller PS5 DualSense  
Arm address 192.168.5.1  
Revision 2026-10-04

Sections 8 (Running on Windows) and 9 (Reference) are developer material and are kept in the repository rather than here: [github.com/k211/DOBOT-NSL](https://github.com/k211/DOBOT-NSL)

# Contents

---

- 01 Safety and the emergency stop
- 02 Status lights
- 03 Unpacking, packing and connecting
- 04 Running and stopping the program
- 05 Joystick controls
- 06 The probe: payload and contact force
- 07 Limits, faults and recovery

## Safety and the emergency stop

Read this before touching anything. The rest of the manual assumes you know where the stop button is.

---

### The arm does not feel you

⚠ This robot is **position controlled**. It moves to wherever the joystick tells it to go and keeps pushing to stay there. It does **not** stop when it meets your hand, the bench, or the phantom. There is no force sensor fitted.

Treat every commanded move as something that will happen regardless of what is in the way.

### The emergency stop

The red mushroom button on a cable from the robot base is the only control that stops the arm unconditionally. Press it and the arm halts all motion and locks.

- It **latches down** when pressed — it stays in until you release it.
- To release, **rotate** the button in the direction of the arrow marked on it.
- Alarms cannot be cleared and the arm cannot be enabled until it is released.
- Keep it within arm's reach of the operator, not buried behind the laptop.

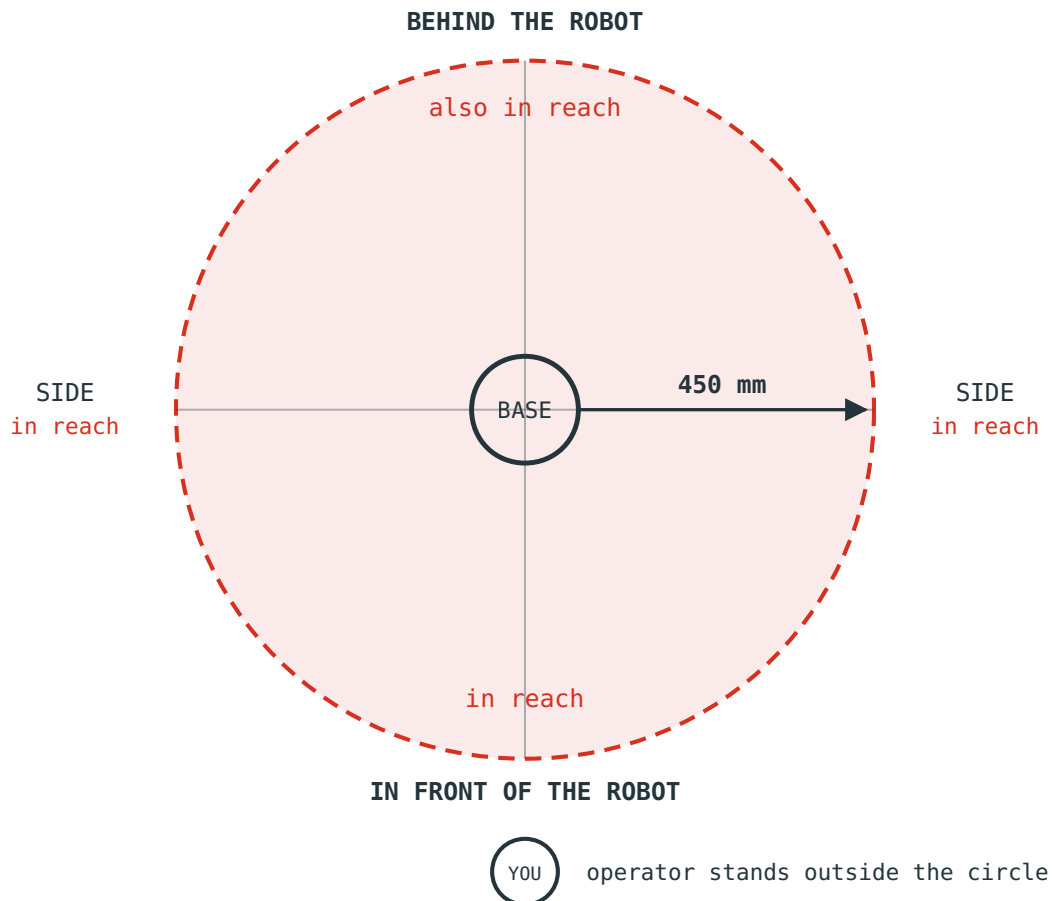
### The deadman principle

Nothing moves unless **L1** is held down. This is deliberate: dropping the controller, letting go in a panic, or walking away all stop the arm.

**Releasing L1 should be your first reflex** — it is faster and gentler than the mushroom button. Save the emergency stop for when the arm is doing something you did not command.

### The workspace

**The robot base is the centre.** The arm can reach **450 mm in every direction** from it — in front, **behind**, to both sides, and upward. It is a full circle around the base, not a zone in front of it.



- **Behind the robot is not safe.** The base can rotate a full  $\pm 360^\circ$ , so the arm can swing round to the back as easily as the front. People standing behind the table are inside the workspace.
- **Mark the full circle** on the table and floor with tape, front and back.
- The **operator stands in front**, outside the circle. Visitors stay outside it on every side.
- Nothing that should not be hit — laptops, the iPad, cups, bags — goes inside the circle. The phantom does, because it has to.

## Rules for the stand

Visitors **may** drive the arm — that is the point of the demonstration — but only under direct supervision, and only within limits.

### Before handing the controller over

1. Show them **L1**: "hold this to move, let go and it stops."
2. Tell them to let go the moment anything surprises them.
3. Keep the controller tethered to your stand, or keep a hand on the cable.

### While a visitor is driving

- **Stay beside them with a hand near the emergency stop.** Supervision means watching the arm, not chatting over your shoulder.
- **Visitors drive in free space only.** *You* make any approach to the phantom — the arm cannot feel contact and will keep pressing. See [§6 Contact force](#).

- Nobody puts a hand inside the workspace while the program is running — 450 mm from the base, **front and back**. [See above](#).
- Keep the phantom, gel and probe cable clear of the arm's path.
- If anyone is unsure what the arm is about to do, **release L1**. That is always safe and always stops it.

## Turbo is an operator control

**Do not mention or demonstrate R1 to visitors.** Turbo multiplies every rate by **1.8**, and the whole reason visitor driving is safe is that the base rate is slow enough to watch and react to. Visitors should finish the demonstration without knowing the button exists.

Keep it for yourself when you need to reposition the arm quickly between visitors — and never for an approach toward the phantom or a person.

If you would rather remove the risk entirely, set `scale_turbo` to `1.0` in `joy_params.yaml` and restart. See §9 (online).

## Things that are safe, and often mistaken for faults

|                                       |                                                              |
|---------------------------------------|--------------------------------------------------------------|
| A clicking sound when the arm enables | Joint brakes releasing. Normal and unavoidable               |
| A fan running constantly              | The control computer lives in the base                       |
| The arm stopping dead mid-move        | Usually a joint limit or singularity. <a href="#">See §7</a> |
| The controller buzzing                | Singularity proximity warning, not contact force             |
|                                       |                                                              |
|                                       |                                                              |
|                                       |                                                              |

## Status lights

A ring of LEDs sits near the base joint and near the wrist. It is the fastest way to know what the arm is doing.

| LIGHT                      | MEANING                                                    | WHAT TO DO                                                               |
|----------------------------|------------------------------------------------------------|--------------------------------------------------------------------------|
| ● <b>Blue, fast flash</b>  | Booting                                                    | Wait 30–60 s                                                             |
| ● <b>Blue, steady</b>      | Powered, <b>not enabled</b>                                | Normal with the program stopped. Ready to launch                         |
| ● <b>Green, steady</b>     | Enabled and idle                                           | Normal just after launch                                                 |
| ● <b>Green, slow flash</b> | Running under remote commands                              | <b>Normal while teleop is live</b>                                       |
| ● <b>Green, fast flash</b> | Jogging, drag mode, or recording                           | Not used by this setup — investigate                                     |
| ● <b>Yellow</b>            | <b>Collision detected</b> — the arm has stopped and paused | Ctrl+C , check nothing is touching the arm, then <a href="#">restart</a> |
| ● <b>Red, steady</b>       | <b>Alarm, or collision detected</b>                        | Stop. <a href="#">Troubleshooting</a> →                                  |

Fast flash is about 5 per second, slow about 1 per second.

## The two you will see all day

|                            |                                         |
|----------------------------|-----------------------------------------|
| <b>Steady blue</b>         | Program stopped, arm resting            |
| <b>Slow-flashing green</b> | Program running, arm under your control |

Anything **red** means an alarm — stop and read [Troubleshooting](#).

Blue, green and yellow are confirmed on this arm: yellow appeared when collision detection tripped during teleop ( `CollisionState 1` ). The red row comes from the manufacturer's table and has not been seen yet — but any red light means an alarm, which is all you need in order to act.

## Unpacking, packing and connecting

The arm travels folded into a **packing posture**, and getting into and out of that posture needs **DobotStudio Pro on the Windows laptop**. Do that first — the Linux laptop is not involved until the arm is already unfolded.

### Three different poses — do not confuse them

| POSE                   | WHAT IT IS                                                                      | HOW YOU REACH IT                                             |
|------------------------|---------------------------------------------------------------------------------|--------------------------------------------------------------|
| <b>Packing posture</b> | A compact fold that fits the foam in the box                                    | DobotStudio Pro → <b>RunTo</b>                               |
| <b>Home pose</b>       | The upright starting posture                                                    | DobotStudio Pro → <b>Home pose</b>                           |
| <b>Our teleop home</b> | <code>[0, -19.24, 117.25, -8.02, -90, 0]°</code> , flange parallel to the bench | Automatic — the controller combo, or <code>go_home.py</code> |

The first two are Dobot's own, reached only through their software. The third is this project's, and is what the arm uses during a demonstration.

### The DobotStudio Pro procedure

**The same procedure both unfolds and folds the arm** — only the final button differs. It runs on the **Windows laptop**, not the Linux one.

1. Connect the **Windows laptop** to the robot by Ethernet.
2. Open **DobotStudio Pro**.
3. Connect to the robot using **EtherCAT**.
4. **Enable** the robot.
5. Go to **Settings → Posture**.
6. **Set the speed below 40%**. Do this before moving anything — the arm folds through a large range and you want time to react.
7. Then, depending on what you are doing:

|                  | BUTTON           | ACTION                                      |
|------------------|------------------|---------------------------------------------|
| <b>Unpacking</b> | <b>Home pose</b> | Click and <b>hold</b> until the arm arrives |
| <b>Packing</b>   | <b>RunTo</b>     | Click and <b>hold</b> until the arm arrives |

1. **Disable** the robot.
2. Disconnect the Windows laptop and move the Ethernet cable to the **Linux laptop**.

**The click-and-hold is a deadman.** The arm moves only while the button is held, and stops the moment you let go. If anything looks wrong, release.

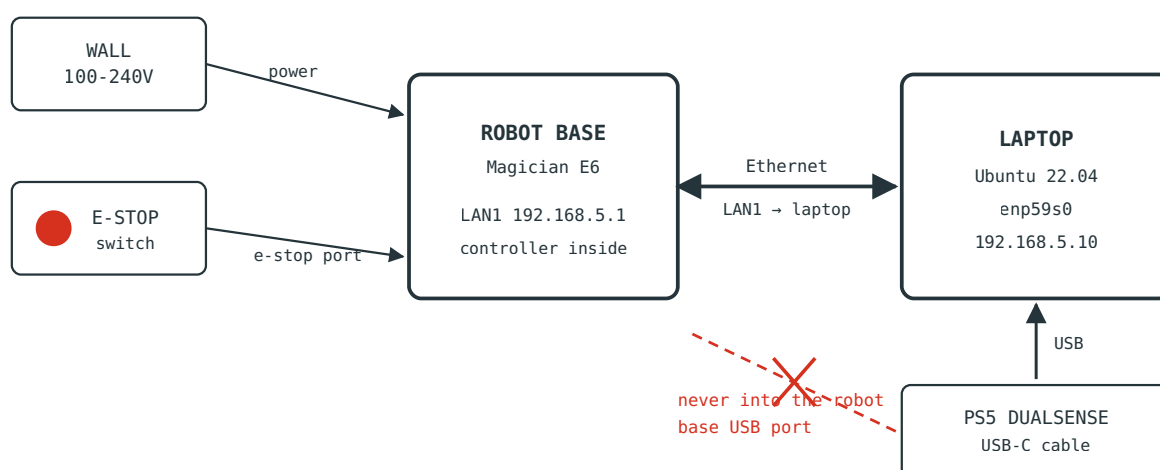
⚠ Keep a hand near the **emergency stop** throughout. The arm is bolted to the bench and the fold sweeps a wide path — check it will not drive into the benchtop, the phantom, or a cable.

## Unpacking

1. Open the box flat on the floor or a sturdy table. The arm weighs **7.2 kg** and is awkward — **use two people**.
2. Lift by the **base and upper arm together**, keeping the folded posture it was packed in. Do not lift it by the forearm or the wrist.
3. **Bolt the base down before doing anything else**. An unbolted arm will tip itself over the moment it moves.
4. Keep the box and foam — you need them to get home again.
5. Connect the cables (below) and power on.
6. Run the **DobotStudio Pro procedure** above, choosing **Home pose**.

⚠ The arm reaches 450 mm in every direction from its base — front, back and sides — and can move its tool at 0.5 m/s. A 7.2 kg arm doing that on an unsecured base will walk off the bench. **Mount it before powering on.**

## The connections



| ORDER | CABLE          | FROM              | TO                                                 |
|-------|----------------|-------------------|----------------------------------------------------|
| 1     | Emergency stop | E-stop switch     | <b>Robot base</b> e-stop port (usually pre-fitted) |
| 2     | Power          | Robot base        | Wall socket — leave switched off for now           |
| 3     | Ethernet       | Robot <b>LAN1</b> | Laptop wired port                                  |
| 4     | USB-C          | PS5 controller    | <b>Laptop</b>                                      |

**The controller plugs into the laptop, never into the robot.** The USB socket on the robot base is for a WiFi dongle or file transfer only — a controller plugged in there does nothing.

LAN1 is the port fixed at `192.168.5.1`. LAN2 is a different subnet and will not work. Connect everything **before** switching on at the wall.

Network configuration for the Linux laptop is in [§4 Running and stopping](#).

---

## Powering the arm on

⚠ **Tap it — do not hold it.** A short press **under 0.5 s** switches the arm **on**. **Holding for 1.5 s is the shut-down gesture.** If you press and hold you will see the light flash and then die when you let go — that is the robot booting and then being told to switch off. Every other button on the robot uses long-press to activate; the power button is the exception.

1. Check the e-stop is **not latched**. If pressed in, rotate to release.
2. Switch on at the wall.
3. **Quick tap** the power button on the base. Finger off immediately.
4. Watch the LED ring: blue fast flash (booting, 30–60 s) → **steady blue**.

**Steady blue is the ready state** — powered and healthy, waiting for the program. See [§2 Status lights](#).

---

## Packing down at the end

1. **Stop the program** with `Ctrl+C` on the Linux laptop. DobotStudio and this software cannot both control the arm.
2. Remove the probe and any tooling from the flange.
3. Move the Ethernet cable to the **Windows laptop**.
4. Run the **DobotStudio Pro procedure** above, choosing **RunTo**.
5. **Disable** the robot in DobotStudio.
6. Power off: **hold** the power button for 1.5 s until the light flashes red fast, then release.
7. Switch off at the wall and disconnect the cables.
8. Unbolt the base.
9. Lift with two people, keeping the folded posture, and lower it into the foam.

⚠ **Do not transport it in any other pose.** The manufacturer requires the packing posture with working brakes before it goes in the box — the brakes are what hold the pose once power is off. Transporting it folded wrongly risks damaging the joints, and the foam will not support it.

---

---

## Running and stopping the program

Everything runs from one terminal on the **Linux laptop**. There is **no timer and no auto-shutdown** — it runs until you stop it.

### 1. Network

Needed after every laptop reboot. The wired adapter carries a static address on the robot's subnet with **no gateway**, otherwise traffic for the robot leaves over WiFi instead.

```
nmcli con up dobot-e6
ping -c3 192.168.5.1          # must reply before going further
```

If the profile is ever missing, recreate it once:

```
sudo nmcli con add type ethernet ifname enp59s0 con-name dobot-e6 \
ip4 192.168.5.10/24 ipv4.never-default yes
```

### 2. Load the environment

```
cd "/home/yz22/Documents/code/Dobot arm"
source setup_dobot.bash
```

### 3. Send the arm to its teleop home

Run this **before** launching. It needs the robot's command port, which the running program holds open — so it will not work once the launch is up.

```
python3 src/dobot_e6_hw/scripts/go_home.py --speed 5
```

It counts down from three before moving, prints joint angles as it goes, and travels slowly. The click as it starts is the joint brakes releasing — normal whenever a disabled arm is enabled.

### 4. Launch

```
ros2 launch dobot_e6_hw real_hw.launch.py robot_ip:=192.168.5.1
```

To pivot rotations on the bare flange instead of the probe, launch `real_hw_flange.launch.py` with the same arguments. [See §5](#).

## What a good startup looks like



Watch for these four lines:

```
[dobot_tcp_node]: RequestControl → 0, {}, RequestControl();  
[dobot_tcp_node]: EnableRobot → 0, {}, EnableRobot();  
[activate_servo]: Servo started – hold L1 and move the sticks to drive.  
[haptic_feedback]: Calibration complete – peak_cond=16.5 LOWER=24.8 HARDSTOP=44.7
```

### Wait for the slow-flashing green light

**Do not touch the controller until the LED ring is flashing green slowly.** That is the arm's own confirmation that it is enabled and accepting remote commands — about 15 seconds after launch.

Then keep your hands off for a further **25 seconds**, until `Calibration complete` appears. That window measures how the arm behaves at rest to tune the rumble warning. Move the arm during it and the warning will be mis-tuned — you will get buzzing at rest, or no warning when you need one.

Allow about **40 seconds** from launch to driving. [More on the lights →](#)

`RequestControl()` is what claims TCP control of the arm. It runs at **every** startup because the grant does not survive a power cycle. No DobotStudio Pro and no Windows machine is needed for this part.

## Stopping

Press **Ctrl+C** in the terminal running the launch and wait for the prompt to come back. The software disables the arm on the way out, so the LED returns to **steady blue**. That is the clean, finished state — there is nothing else to shut down.

## Homing while the program is running

Use the controller: hold **L1 + L2 + R2** for one second.

`go_home.py` only works with the launch stopped, so it is for before you start or after an error — not during a demonstration.

## Health checks (optional)

Only needed if something looks wrong. Run in a **second** terminal with the program still going:

```
source setup_dobot.bash
ros2 topic echo /servo_node/status      # 0 = no warnings
ros2 topic echo /joint_states --once    # do the angles match the real pose?
ping -c3 192.168.5.1                    # network alive?
ls /dev/input/js0                       # controller present?
```

---

---

## Joystick controls

**Hold L1 or nothing happens.** Every direction is described from the operator's point of view, standing in front of the arm and facing it.



**HOME:** hold L1 + L2 + R2 for 1 s, then release L1 and press it again to drive

Photo: Chris Woodrich, CC BY-SA 4.0, via Wikimedia Commons; annotations added

## What each control does

| CONTROL               | MOTION                                       | RATE      | FEEL                 |
|-----------------------|----------------------------------------------|-----------|----------------------|
| <b>L1</b>             | <b>Deadman</b> — hold it or nothing moves    | —         | on / off             |
| <b>D-pad</b> ◀▶       | Move to your <b>left / right</b>             | 0.03 m/s  | full rate or nothing |
| <b>D-pad</b> ▲▼       | Move <b>away from / toward</b> you           | 0.03 m/s  | full rate or nothing |
| <b>Left stick</b> ▲▼  | Move <b>up / down</b>                        | 0.03 m/s  | proportional         |
| <b>Right stick</b> ▲▼ | <b>Yaw</b> — twist about vertical            | 0.3 rad/s | proportional         |
| □ / ○                 | <b>Pitch</b> — tilt the probe nose           | 0.2 rad/s | full rate or nothing |
| △ / ×                 | <b>Roll</b> — rotate about the probe axis    | 0.2 rad/s | full rate or nothing |
| <b>R1</b> (with L1)   | <b>Turbo</b> — <i>operator only, see §1</i>  | ×1.8      | hold                 |
| <b>L1 + L2 + R2</b>   | <b>Home</b> — return to the safe pose        | slow      | hold 1 s             |
| L2 or R2 alone        | Nothing                                      | —         | —                    |
| PS button             | Nothing — <b>not visible to the software</b> | —         | —                    |

The PS button is unusable: the index exists in the controller's button array but the press never reaches the software, because something upstream grabs it. It was deliberately left unused.

## Rotations pivot on the probe

Roll, pitch and yaw turn the arm **about the probe face**, not about the bare flange — the point stays where it is and the arm swings around it. That point is **0 mm X, 65 mm Y, 98 mm Z** from the flange. Translations move that same point.

Two launch files, identical apart from the pivot:

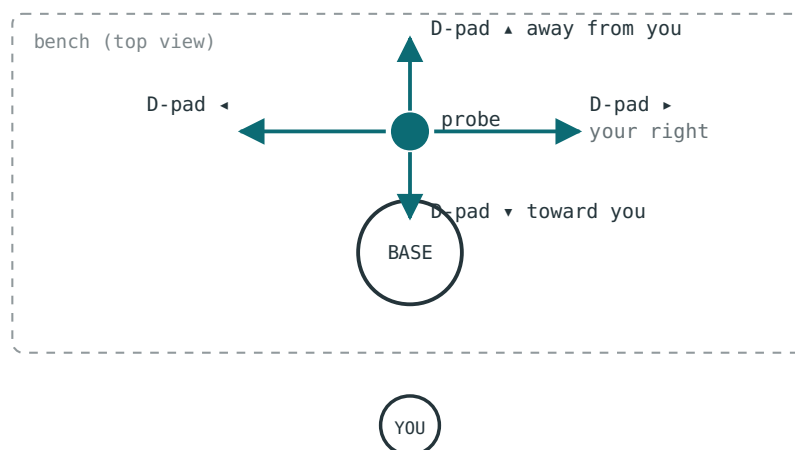
| LAUNCH FILE                           | ROTATIONS PIVOT ON                 |
|---------------------------------------|------------------------------------|
| <code>real_hw.launch.py</code>        | <b>the probe</b> , (0, 65, 98) mm  |
| <code>real_hw_flange.launch.py</code> | <b>the bare flange</b> , (0, 0, 0) |

To try another pivot without editing anything, pass it in metres:

```
ros2 launch dobot_e6_hw real_hw.launch.py robot_ip:=192.168.5.1 tcp_x:=0.065 tcp_y:=0.0
tcp_z:=0.098 # the earlier (65, 0, 98)
```

Homing is the same in both: it goes to fixed joint angles and does not use the pivot.

## Directions are from where you stand



Stand **in front of the arm, facing it**. The D-pad then moves the probe the way the arrow points, from your point of view — so your left is the robot's right.

If you walk round to the side of the bench, the mapping no longer matches your body. Move back to the front before driving.

## Homing

Hold **L1 + L2 + R2 together for one second**. The arm pauses, moves gently to the safe pose, and stops.

**Afterwards, teleop stays disabled until you release L1 and press it again.** This is deliberate — it stops a stick you are still holding from flinging the arm the instant homing finishes. If the arm seems dead after homing, this is almost always why.

Homing moves joint by joint, so it works even from a pose that ordinary driving cannot escape. But it takes the **direct route with no collision checking** — clear the phantom out of the way and keep a hand near the e-stop.

## The rumble

The controller vibrates as the arm approaches a **kinematic singularity**. Stronger buzz means closer. [See §7 →](#)

⚠ **The rumble is not contact force.** No force sensor is fitted. The vibration describes the arm's *geometry*, not how hard the probe is pressing. You will feel nothing when the probe touches down.

## Re-mapping

The mapping lives in `src/dobot_e6_bringup/config/joy_params.yaml`. Edit and restart the program — no rebuild needed.

Every index in that file was **measured** on this specific controller, not inferred from documentation. If you change controllers, measure again rather than assuming: the kernel and the ROS driver disagree about D-pad sign, and an earlier version of this project had a trigger mapped so that the arm climbed at full speed the instant the deadman was pressed.

---

---

---

## The probe: payload and contact force

The two questions that matter most for scanning: what changes when you fit the probe, and what happens when it presses on the phantom.

### Fitting the probe

#### Mechanical — stay under 750 g

The E6's rated maximum load is **750 g**, and that budget covers **everything** you bolt on: probe, adapter plate, bolts, and any cable weight the arm actually carries. A typical clinical probe is 200–500 g, so there is room — but weigh the assembled tool rather than guessing.

- Mount to the **four M6 threaded holes** in the end flange.
- Use the 6 mm locating hole with a dowel pin if you need repeatable alignment.
- **Strain-relieve the probe cable** to the forearm. A cable that snags mid-scan drags the arm off course, and the arm will not notice.
- Keep the probe's mass close to the flange. Weight hung far out loads the wrist joints much harder than the same weight held close.

#### Software — tell the controller about the load

The probe payload is declared to the controller every time the arm is enabled:

|                           |                                                              |
|---------------------------|--------------------------------------------------------------|
| Mass                      | <b>355 g</b> (probe + holder)                                |
| Centre of mass            | <b>X 0 mm, Y 65 mm, Z 50 mm</b> , in the flange (tool) frame |
| Probe face (teleop pivot) | X 0 mm, Y 65 mm, Z 98 mm — a different point, see §5         |

These live in `src/dobot_e6_hw/scripts/home_pose.py` ( `PROBE_PAYLOAD_KG` , `PROBE_COM_MM` ). If the probe or holder changes, re-weigh it and update them.

**Running without the probe?** Tell the software, so the declared load matches what is on the flange:

```
python3 src/dobot_e6_hw/scripts/go_home.py --speed 5 --no-probe
ros2 launch dobot_e6_hw real_hw.launch.py robot_ip:=192.168.5.1 probe:=false
```

Use the same setting for both commands.

#### What if the payload is declared but the probe is not fitted?

Nothing dangerous, but the controller's model is wrong by the weight of the probe. In practice:

- **The arm still moves correctly.** It is position controlled, so it still goes where the joystick sends it. The error in its internal force model is about **3.5 N** (355 g × gravity) at roughly 80 mm from the flange — small next to what the arm can carry.
- **Collision detection may trip when nothing has hit anything.** The controller expects to work harder than it does, and reads the difference as an outside force pushing the tool **upward**. That

can raise a collision alarm (red light, arm stops), most likely during quick moves and when the wrist is tilted.

- **It cannot detect that the probe is missing.** The automatic load check is switched off on purpose ( `PAYLOAD_CHECK = 0` ), because a false trip of that check would disable the arm mid-demonstration. So the mistake will not be flagged for you.
- The **opposite mistake** — probe fitted, payload not declared — has the same effect in the other direction.

**If you see collision alarms with nothing touching the arm, check that the probe setting matches what is on the flange.** Then do the full restart in §7.

These effects come from how the controller's model works; the size of the collision threshold on this arm has not been measured, so how often a false alarm actually happens is unknown.

## Re-home after fitting

Stop and restart the program once the probe is on. The haptic calibration measures the arm at rest, and it should do that with the final tool attached.

## Contact force on the phantom

⚠ **The arm does not yield.** This is the most important thing in this manual.

The arm is **position controlled**. When the probe meets the phantom it does **not** sense the contact and does **not** back off. It keeps driving toward the position your joystick commanded, pressing harder the further you push past the surface. It behaves like a clamp, not like a hand.

## What actually happens when you press down

1. You hold **L1** and push the left stick down. The arm descends at 0.03 m/s.
2. The probe touches the phantom. **Nothing changes** — no resistance is felt by the software, no rumble, no slowing.
3. Keep pushing and the arm keeps descending, deforming the phantom and loading the probe.
4. Eventually either the controller's **collision detection** trips and raises an alarm, or it does not — and the force keeps climbing.

Step 4 is the uncomfortable one. Collision detection is a safety backstop with a threshold, designed to notice a crash, **not** to regulate scanning pressure.

## How much force can it generate?

**Dobot does not publish a maximum force figure for the E6.** The 750 g load rating is *not* a force limit — it describes what the arm can carry and accelerate, and a position-controlled arm pressing into a rigid object can generate substantially more than the weight it is rated to lift.

Treat the pressing force as **unknown and potentially much higher than you expect**.

## How to scan safely, given all of the above

- **Approach in small taps.** Tap the stick down, release, look. Do not hold it down and watch.
- **Release L1 the instant the probe touches.** That freezes the arm.

- **Judge pressure by eye** — watch how far the phantom surface deforms. That is your only feedback.
- **Never put a hand between probe and phantom** to check contact.
- **Do not use turbo** for approach moves.
- Use a **compliant phantom** if you have the choice. Against something rigid, a small over-travel becomes a large force very quickly.

## If you need real force control

Proper scanning pressure needs a **wrist force/torque sensor**. The software already has a contact-force rumble cue written and waiting — it needs a sensor publishing on `/ft_sensor/wrench`, and none is fitted. With that hardware the arm could hold a target *force* instead of a target *position*.

That is the correct engineering answer for scanning, and it is a hardware addition, not a software setting.

---

---

---

# Limits, faults and recovery

What to do when the arm stops, misbehaves, or shows a red light. Start at the top — most "faults" are the arm behaving correctly at a limit.

## First: a stopped arm is usually not a fault

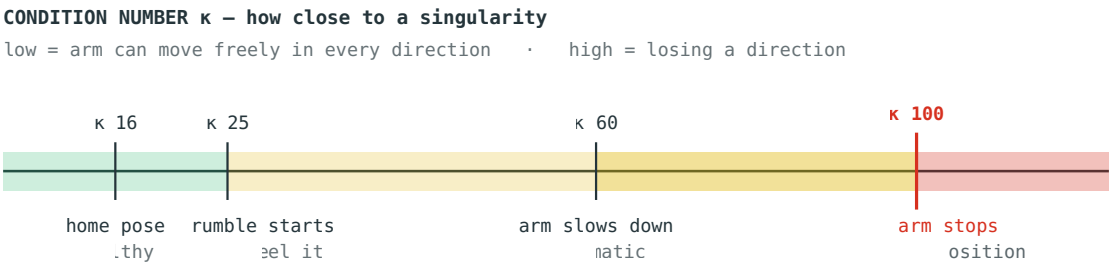
✓ When the arm reaches a joint limit or a singularity it **stops moving and holds position**. It does not shut down, power off, drop, or raise an error. Motion simply stops being commanded. **No restart is needed** — drive back out the way you came.

Check this before assuming something is broken. The giveaway is a **steady or slow-flashing green** light and no error in the terminal.

## Singularities

A singularity is a pose where the arm loses the ability to move in some direction, typically when joints line up. Near one, small tool movements demand enormous joint speeds, so the software intervenes early.

Closeness is measured by the **condition number ( $\kappa$ )**: low is healthy, high is dangerous.



| K   | WHAT HAPPENS                                                                       |
|-----|------------------------------------------------------------------------------------|
| ~16 | The home pose. Healthy                                                             |
| ~25 | <b>Controller rumbles.</b> Re-measured at every launch, so the exact figure varies |
| 60  | <b>The arm automatically slows down</b>                                            |
| 100 | <b>The arm stops</b> and holds position                                            |

You get three escalating warnings before anything stops.

## Joint limits

The software stops commanding motion **0.1 radians (about 5.7°) before** a joint reaches its limit, so the arm never runs into its own hard stops. You will also feel rumble as a joint nears its limit.

| JOINT       | TRAVEL          | JOINT      | TRAVEL          |
|-------------|-----------------|------------|-----------------|
| J1 base     | $\pm 360^\circ$ | J4 wrist 1 | $\pm 160^\circ$ |
| J2 shoulder | $\pm 135^\circ$ | J5 wrist 2 | $\pm 173^\circ$ |
| J3 elbow    | $\pm 154^\circ$ | J6 flange  | $\pm 360^\circ$ |

## Getting unstuck

### 1. Release L1.

2. Press it again and drive **in the opposite direction** to the one that got you stuck.

3. If that does not free it, hold **L1 + L2 + R2** for a second to home the arm. Homing moves joint by joint, so it works from poses that ordinary Cartesian driving cannot escape.

⚠ Homing takes the **direct joint-space route with no collision checking**. From a twisted wrist that is harmless, but if the arm is somewhere awkward, watch the first second with a hand near the e-stop, and move the phantom clear first.

## Symptom table

| SYMPTOM                                                                         | MOST LIKELY CAUSE                                                                                                                                                                                                        | FIX                                                                                                                                                               |
|---------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Arm does not move when L1 is held                                               | You just homed it — teleop is deliberately disarmed                                                                                                                                                                      | <b>Release L1 and press it again.</b> By design                                                                                                                   |
| Still does not move; LED green                                                  | Joint limit or singularity reached                                                                                                                                                                                       | Drive the other way, or home it — see above                                                                                                                       |
| Arm stops part-way through a move                                               | Same. Not a fault                                                                                                                                                                                                        | Drive the other way                                                                                                                                               |
| Every command rejected, <code>Control Mode Is Not Tcp</code>                    | DobotStudio is connected and holding the arm                                                                                                                                                                             | Close DobotStudio completely, then restart. The software claims control automatically at startup                                                                  |
| Launch hangs with no output                                                     | Wrong IP, robot off, or no network                                                                                                                                                                                       | <code>ping -c3 192.168.5.1</code> . If it fails, <code>nmcli con up dobot-e6</code>                                                                               |
| Constant rumble, arm clearly fine and not moving                                | The arm is silently refusing commands, so the software's idea of its pose drifts                                                                                                                                         | Look for <code>ServoJ rejected</code> in the terminal. Full restart                                                                                               |
| Rumbles at rest right after startup                                             | The arm was moved during the 25 s calibration window                                                                                                                                                                     | Restart and keep hands off for the first 40 s                                                                                                                     |
| Yellow light while moving, stays yellow after <code>Ctrl+C</code>               | Collision detection tripped: the arm is paused. <code>Ctrl+C</code> does not clear it                                                                                                                                    | Check nothing is touching the arm, then repeat start-up steps 4–5. <code>go_home.py</code> clears the alarm and flushes the paused motion itself                  |
| Red light                                                                       | Alarm                                                                                                                                                                                                                    | Full restart. If it returns immediately, power-cycle the arm                                                                                                      |
| Wrist or probe drops slightly when the program stops or the robot is turned off | Normal: the motors let go a fraction of a second before the joint brakes grip, and the probe's weight pulls the wrist down in that gap. The payload setting cannot prevent it — it only applies while the arm is enabled | Support the probe by hand when stopping or turning off. If it drops more than a few millimetres, or keeps sinking, stop using the arm and have the brakes checked |
| Controller does nothing at all                                                  | USB not detected                                                                                                                                                                                                         | Unplug and replug. Check <code>ls /dev/input/js0</code> . Restart the program                                                                                     |
| Arm drifts with nobody touching anything                                        | Stick drift on a worn controller                                                                                                                                                                                         | Release L1. Restart; if it persists the controller needs replacing                                                                                                |
| Collision alarms with nothing touching the arm                                  | Probe setting does not match what is on the flange — probe declared but not fitted, or the reverse                                                                                                                       | Match <code>probe:=true/false</code> to the flange, then full restart. <a href="#">See §6</a>                                                                     |

## Does it need a restart?

| SITUATION                               | RESTART?                                                   |
|-----------------------------------------|------------------------------------------------------------|
| Singularity or joint limit reached      | <b>No</b> — drive back out                                 |
| Rumbling constantly but the arm is fine | <b>No</b> — home it                                        |
| Arm dead right after homing             | <b>No</b> — release L1 and press it again                  |
| Emergency stop pressed                  | <b>Yes</b> — release it first                              |
| Red light / alarm state                 | <b>Yes</b>                                                 |
| Arm ignores the controller entirely     | <b>Yes</b>                                                 |
| Network dropped, ping fails             | <b>Yes</b> — and re-run <code>nmcli con up dobot-e6</code> |

## The full restart



```
# 1. Ctrl+C in the terminal running the launch
# 2. release the emergency stop if it is latched (rotate it)

nmcli con up dobot-e6
ping -c3 192.168.5.1

cd "/home/yz22/Documents/code/Dobot arm"
source setup_dobot.bash
python3 src/dobot_e6_hw/scripts/go_home.py --speed 5
ros2 launch dobot_e6_hw real_hw.launch.py robot_ip:=192.168.5.1

# 5. wait for the slow-flashing green light, then 25 s more for calibration
```

If a red light persists after a full restart, power-cycle the arm: **hold** the power button 1.5 s to switch off, wait ten seconds, then **tap** once to switch on.

## Reading the launch output

| LINE                                | MEANS                                                                         |
|-------------------------------------|-------------------------------------------------------------------------------|
| <code>RequestControl → 0,{}</code>  | TCP control claimed. Good                                                     |
| <code>EnableRobot → 0,{}</code>     | Arm enabled. Good                                                             |
| <code>Servo started</code>          | Ready to drive                                                                |
| <code>Calibration complete</code>   | Rumble threshold tuned. Safe to touch the controller                          |
| <code>ServoJ rejected: ...</code>   | <b>The arm is refusing motion.</b> Check mode and alarms                      |
| <code>RequestControl refused</code> | Something else holds the arm — close DobotStudio                              |
| <code>Robot PAUSEd</code>           | Controller stuck in pause; the software sends <code>Stop()</code> to flush it |

Diagnostic commands are in [§4 Running and stopping](#).